Get It. · Technical Writeup

> *"What I cannot create, I do not understand." · Richard Feynman, last blackboard at Caltech, February 15, 1988.*

A developer-oriented walk through the codebase. The goal is that a contributor who has never read the project before can finish this document and confidently navigate the source, change a piece of behaviour, or replace a layer.

Mental model

Get It. is a desktop app for studying a single PDF at a time. Drop a file. Two pipelines fire in parallel from upload. The first one ships visualizations inline next to the text so the document is immediately easier to read. The second one builds a concept graph of the same document and scores the student's mastery on four orthogonal axes as they interact with four study tools (chat, flashcards, forced-choice quizzes, Feynman with a curious child). Every interaction lands in one append-only journal on disk. One evaluator agent reads that journal and updates the four-axis scores after every completed session.

```
upload  ──┬──► pdfjs-dist extracts text + glyph bboxes per page
          │
          ├──► visualizer pipeline
          │     ├─ per-page concept-detection agent  →  DetectedConcept[] with anchor strings
          │     └─ per-tag visualization-spec agent  →  3d / 2d-anim / formula / graph / 2d-text spec
          │                                            (server-side syntax preflight + client-side
          │                                             runtime repair loop on sandbox crashes)
          │
          └──► knowledge-graph pipeline
                ├─ kg-build agent (one-shot)            →  6–25 concept nodes + typed edges + global note
                └─ kg-evaluate agent (debounced)        →  per-node {memory, comprehension, structure,
                   ◄── work-context journal                 application} 0–100, monotone non-decreasing
                   ◄── document text
```

Persistent state is a tree of plain JSON files under one OS-native user-data directory. There is no database, no hosted backend, no shared key pool, no analytics SDK. Every model call is an `@openai/codex-sdk` invocation against the end user's own ChatGPT account.

Code map

The product is a single Next.js 16 application wrapped in a small Electron shell.

```
electron/                  desktop shell + setup wizard + auto-update
  main.js                  single-instance lock, data dir resolution, server spawn
  setup.js                 Codex CLI install + OAuth wizard window
  updater.js               GitHub Releases poll, in-app installer flow
  update-window/           wizard HTML/JS for the update modal
  wizard/                  wizard HTML/JS for the first-launch wizard
  codex-bin/<triple>/      bundled Codex CLI per platform/arch
  preload*.js              context-isolated preload bridges

app/                       Next.js App Router pages + API routes
  page.tsx                 upload home
  library/                 catalog of every opened PDF
  viewer/[docId]/          per-document viewer (PDF + right pane)
  api/
    upload/                pdfjs extraction + new docId or sample reuse
    analyze-pdf/           legacy single-shot detection (preserved for tests)
    tags/[docId]/          server-owned tag store: GET / POST active-tag / etc.
    jobs/detect/[docId]    POST → kicks the concept-detection job for a doc
    jobs/viz/[docId]       POST → kicks per-tag viz-spec generation
    chat/[docId]           POST → chat turn (triggers scheduleEvaluation)
    flashcards/[docId]     POST generate / rate / end (triggers scheduleEvaluation)
    quizzes/[docId]        POST generate / answer / end (triggers scheduleEvaluation)
    feynman/[docId]        POST start / explain (triggers scheduleEvaluation)
    kg/[docId]/            build / state / evaluate
    work-context/[docId]   download the journal as JSON
    codex/health           process-local CodexError mailbox (banner polls this)
    codex/account|logout   surface ChatGPT account + sign-out

components/                React UI (orchestrators + scene renderers + tag UI)
  RightPane/               mode dropdown + the four tool views + KG view
  Visualizer/              3D / 2D / formula / graph / text renderers + sandbox
  PdfViewer.tsx            pdf.js viewer with overlay tag pills
  CodexHealthBanner.tsx    error banner + countdown + re-connect

lib/                       framework-agnostic helpers
  codex.ts                 the one and only LLM transport: runJson + error classifier
  agents/                  per-agent prompt builders (detect, viz)
  kg.ts / kg-runner.ts     KG persistence + build/eval runners + scheduler
  store.ts                 doc cache + filesystem persistence
  paths.ts                 the OS-native data-dir resolver
  pdf-extract.ts           pdfjs-dist text + bbox extraction
  schemas.ts schemas-kg.ts JSON schemas for every agent
  work-context*.ts         journal storage + evaluator summary
  viz-runtime.ts           the `new Function` sandbox compiler
  config.ts                runtime-mutable settings + env defaults
```

The agent layer

Every call to OpenAI funnels through one helper at `lib/codex.ts` → `runJson(prompt, outputSchema, opts)`. The helper:

1. Lazily initialises one `Codex` client per process.
2. Starts a fresh thread with `sandboxMode: "read-only"`, `approvalPolicy: "never"`, `skipGitRepoCheck: true`, and an explicit working directory under `<DATA_DIR>/codex-scratch`. The renderer never sees a turn that escaped its own working dir.
3. Runs the turn against the supplied JSON Schema, retries once on parse failure, and returns the typed result.
4. Catches every throw, classifies it into `auth_lost` / `rate_limit` / `binary_missing` / `generic`, and writes the result into a process-local **health mailbox**. The renderer polls `/api/codex/health` to render a banner. Rate-limit retry deadlines are extracted from the error message when present.
5. Short-circuits future calls while a rate-limit window is still active so a chatty UI cannot burn a hundred wasted calls.

Nine prompts live behind that one helper:

WHERE	WHAT IT RETURNS	SCHEMA
`lib/agents/detect.ts`	`DetectedConcept[]` for a single page	`detectionSchema` in `lib/schemas.ts`
`lib/agents/viz.ts`	Per-tag visualization spec (one of five renderer types)	`vizSchemaFor(type)` in `lib/schemas.ts`
`lib/kg-runner.ts → BUILD_SYSTEM`	The graph: 6–25 nodes, typed edges, global note	`kgBuildSchema` in `lib/schemas-kg.ts`
`lib/kg-runner.ts → EVALUATE_SYSTEM`	Per-node updates {memory, comprehension, structure, application} + notes	`kgEvaluateSchema`
`app/api/chat/[docId]/route.ts`	One assistant reply	`chatReplySchema`
`app/api/flashcards/[docId]/route.ts`	4–10 Q / A cards	`flashcardsGenerateSchema`
`app/api/quizzes/[docId]/route.ts`	4–8 MCQs with one correct option and three distractors	`quizGenerateSchema`
`app/api/feynman/[docId]/route.ts → CHILD_SYSTEM`	One curious-child prompt	`feynmanChildPromptSchema`
`app/api/feynman/[docId]/route.ts → SUMMARY_SYSTEM`	End-of-session honest summary	`feynmanSummarySchema`

There is no god-prompt and no client-side JSON-Schema validation. Every agent reply arrives as a typed TypeScript object the rest of the code can use without defensive parsing.

The visualizer pipeline

The pipeline that ships time-to-value: tags appear inline the instant detection returns, the right pane fills in as each per-tag agent finishes.

Server-side jobs. Detection and per-tag viz generation are not renderer loops. Both are first-class **server-side jobs**, singleton-per-doc, idempotent, running inside the Next process. The detection job walks unanalysed pages with concurrency 3 and persists each batch of new tags to `<DATA_DIR>/docs/<docId>/tags.json` as it goes. The viz job picks the next tag whose state is `generating: true`, runs the per-type agent at concurrency 4, and persists the spec back to the same file. The viewer is a *consumer*: it polls `GET /api/tags/<docId>` every 1.5 s while any job is in flight, fires `POST /api/jobs/viz/<docId>` on a user click or a sandbox runtime-error report, and only ever updates the *active-tag selection* on the server. The active selection is the lone field the client can write; everything else is server-owned, so a concurrent client navigation cannot overwrite mid-flight detection or generation.

Reopening a doc from the Library weeks later therefore restores the exact tag layout, viz specs, and analysed-pages set without re-detection. Library badges poll the same source so they stay live across the whole catalog with no extra plumbing.

Five renderer types. `lib/agents/viz.ts` routes by `VizType`:

- **`3d`**. The agent emits a JavaScript function body that `components/Visualizer/ThreeDView.tsx` executes with `{ THREE, scene, camera, renderer, controls, group }` in scope. The viewer auto-frames the molecule with a bbox and auto-rotates the group.
- **`2d-anim`**. Same shape, but the function body returns an object with `draw(ctx, width, height, time, dt)` and runs every frame on a Canvas2D context.
- **`formula`**. A headline LaTeX line plus 2–6 derivation steps with one-sentence explanations; rendered with KaTeX.
- **`graph`**. A `chart_type` (function / points / bars / lines) plus a JSON-string `data_json`; plotted on a Canvas.
- **`2d-text`**. Title plus caption plus markdown body plus citation list. Used for legal articles, named papers, and authoritative quotations. Web search is enabled only for this type.

The sandbox. `lib/viz-runtime.ts → compileFn` wraps each LLM-emitted function body in an IIFE that shadows the dangerous globals (`window`, `document`, `fetch`, `XMLHttpRequest`, `WebSocket`, `Function`, `eval`, `localStorage`, `sessionStorage`, `require`, `Worker`, `WebAssembly`, `process`, `globalThis`) as

`undefined` parameters before the inner function runs. The boundary is a defense against LLM mistakes, not against adversarial input: the user is running their own Codex account against their own PDFs.

Repair loop. When the sandbox throws inside `ThreeDView`'s `setup_code` or in a `2d-anim` `draw`, the viewer reports the error string back to the server, which hands it to Codex as repair context (the broken `setup_code` + the captured error message) and asks for a corrected JSON object that compiles and runs end-to-end. The user sees "repairing, attempt N of M" instead of red text. Server-side syntax pre-flight via `new Function(...)` catches truncated bodies before they ever leave the route.

The knowledge-graph pipeline

This is the layer that turns Get It. from a viewer into a measurement instrument. Two agents, one persistence file, one queue.

`kg-build` runs once per document at upload time. The system prompt asks for 6–25 concept nodes the student would actually need to master (not a glossary), typed edges (prerequisite / composition / causal / specialisation / contrast), and a short global note that the viewer prints above the graph. Output is written to `<DATA_DIR>/docs/<docId>/kg.json` with `status: "ready"`. Rate-limited or auth-lost failures leave the KG in `status: "building"` and re-fire on a setTimeout once the deadline passes; the badge keeps spinning until the next attempt picks up cleanly.

`kg-evaluate` is the four-axis rubric. Every node carries four 0–100 scores:

AXIS	WHAT IT MEASURES	STRONGEST SIGNAL
memory	recall over time	flashcard ratings (1–4), quiz correctness on definitional questions, recall references in chat
comprehension	understanding in the student's own words	original metaphors in chat, plain-language Feynman explanations, distractor-rejection in quizzes
structure	grasp of how concepts connect	multi-step reasoning that bridges concepts, references to prerequisites, sibling discrimination
application	transfer to new cases	original examples, edge cases, novel problem solving, applied-tier quiz answers

The evaluator sees the entire work-context journal (compacted and timestamped via `summariseForEvaluator`), the current graph with previous scores, and the document's own page text. Its system prompt enforces three rules: scores are **monotone non-decreasing**, *quantity does not entitle a score*, and concepts with no observable evidence stay at their previous level. The runtime enforces the monotone rule with a clamp on every update (`clampMonotone` in `lib/kg-runner.ts`) so a chatty interaction cannot accidentally erase prior evidence even if the agent disregards its own instruction.

Scheduling. Each evaluator pass is one Codex turn at medium effort. The chat tool is chatty by definition, so we run a per-doc queue with at most one in-flight pass and one pending. Every tool route fires `scheduleEvaluation(docId)` and returns immediately. The client polls `/api/kg/[docId]/state` (which exposes the live `evaluating` flag), accelerating to 2.5 s while the agent is working and slowing to 6 s when idle. The badge in the top tab bar reads "Building graph", "Evaluating", "No evaluations yet", or "Synced 12 s ago" depending on what the queue is doing.

A rate-limit hit inside an evaluator pass schedules a `setTimeout` for `retryAt + 500 ms` that re-fires `scheduleEvaluation`. No user action needed; the graph keeps catching up on its own.

The four study tools and the work-context journal

The four tools are deliberately small and deliberately different. Each provides a distinct evidence type.

- **Chat.** Multi-turn, multi-thread, scoped to one document. The knowledge-graph node list and a 30 KB document excerpt are injected as system context. Each assistant reply triggers a debounced KG re-evaluation.
- **Flashcards.** Open-recall under self-grade. The student picks a topic (or "all"), Codex generates a 4–10 card deck, the student optionally types their answer, reveals, and self-grades 1–4 (Again / Hard / Good / Easy, the FSRS convention). Ratings are recorded per card; closing a deck triggers an evaluator pass.

- **Quizzes.** Forced-choice discrimination. Codex generates a 4–8 question multiple-choice quiz; each item carries one correct option and three plausible distractors picked to expose the confusion a student would actually trip on. The server **shuffles the options** at generation time with `crypto.randomInt`-driven Fisher–Yates so the agent's positional bias (the model tends to put the right answer at index 0) does not leak to the UI. The student picks, gets immediate feedback with a one-sentence explanation, and the quiz ends with a score summary.
- **Feynman.** The agent plays a curious eight-year-old who asks 3 to 4 short, pointed questions. The student is forced into the role of the teacher. After the last turn a separate summary call writes a 3–6-sentence honest read of where the explanation held and where it broke down. The session is bounded so the data stays usable for the evaluator and the student does not drift.

Behind all four sits one artifact: the **work-context JSON**, one file per doc on the server, append-only by convention. Every chat message, every card rating, every quiz answer, every Feynman turn lands here with a UTC timestamp. It is the file the evaluator reads, the file the student can download from the right-pane menu, and by design the only thing the system needs to remember about a study session. Backwards-compatible loading (`loadWorkContext`) back-fills any array that did not exist when the doc's journal was first written, so quizzes added in v1.1.0 work cleanly against pre-quiz journals from v1.0.0.

Persistent state and the types-split pattern

Filesystem-backed under one OS-native data directory per user, resolved once in `lib/paths.ts`.

OS	PATH
macOS	`~/Library/Application Support/get-it/`
Windows	`%APPDATA%\get-it\`
Linux	`~/.local/share/get-it/`

Or whatever the Electron main pinned via the `GETIT_DATA_DIR` environment variable. Layout:

```
docs.json                                  # top-level catalog
docs/<docId>/source.pdf                    # original bytes
docs/<docId>/meta.json                     # { id, filename, uploadedAt, numPages, lastOpenedAt }
docs/<docId>/extracted.json                # cached pdfjs-dist output (text + bboxes per page)
docs/<docId>/tags.json                     # server-owned visualizer tags + viz specs
docs/<docId>/workctx.json                  # the journal: chats / flashcards / quizzes / feynman
docs/<docId>/kg.json                       # the knowledge graph + per-node scores
codex-scratch/                             # Codex CLI's per-call working dir
logs/                                      # embedded server stderr
settings.json                              # auto-generate, max-repair-attempts
```

Cheap, recoverable, OS-agnostic, and a clear seam to lift to a hosted backend if we ever want to.

Types-split pattern. Modules are split into pure-TS `*-types.ts` files (no `node:fs` imports) and storage helpers in `*.ts` that do touch the filesystem. Next.js bundles a transitively-imported module into the client when *any* type from it is referenced, including a bare `import type {}`. Splitting types into a node-free file is the only way to keep `lib/kg.ts` and `lib/work-context.ts` server-only without poisoning the browser bundle. The comments in those files say so explicitly.

Settings. `lib/config.ts` reads env defaults for the two runtime-mutable settings (`NEXT_PUBLIC_AUTO_GENERATE_VIZ`, `NEXT_PUBLIC_MAX_VIZ_GEN_RETRIES`) and persists overrides to `<DATA_DIR>/settings.json`. The dynamic localhost port the packaged app binds to changes on every launch, so anything cookie- or localStorage-scoped to the origin would forget the user's choice; a plain file in the user-data dir is the only thing that survives a restart. A change broadcasts a `getit:settings` window event so other pages on the same renderer react without polling.

Resilience to Codex outages

Every Codex call funnels through `runJson` in `lib/codex.ts`. That helper classifies failures into four kinds and writes the latest one into a process-local **health mailbox**.

KIND	TRIGGER	UI BEHAVIOUR
`auth_lost`	401 / token revoked / "sign in" message	Banner + "Re-connect" button re-opens the desktop setup wizard
`rate_limit`	429 / "try again in N" / 5-hour / weekly window phrases	Banner with live countdown to `retryAt`; auto-disappears when the deadline passes
`binary_missing`	Codex binary not found at the resolved path	Banner + button to re-install via the wizard
`generic`	Anything else	Banner with the raw message

Two things hang off the mailbox:

1. **The in-app banner.** The renderer polls `/api/codex/health` (fast cadence while there is an active problem, slow cadence otherwise) and renders `components/CodexHealthBanner.tsx`. The countdown updates locally so the banner stays smooth between polls.
2. **The kg-evaluator queue.** Hitting a rate-limit inside an evaluator pass schedules a `setTimeout` for `retryAt + 500 ms` that re-fires `scheduleEvaluation(docId)`. The build agent does the same: it leaves the KG in `status: "building"` instead of erroring out so the badge keeps spinning and the next attempt picks up cleanly. Tool routes (chat / flashcards / quizzes / Feynman) preserve the work-context journal up to the failure point so the student can re-send the same action once the banner clears: no lost messages, no orphan card ratings, no half-finished Feynman session.

`runJson` also short-circuits future Codex calls while a rate-limit window is still active. A chatty UI cannot burn a hundred wasted calls hoping the next one succeeds.

Bring-your-own-account as an architectural choice

The decision to drive every agent through the **user's own ChatGPT login over the official Codex CLI** is the choice that shapes the whole product. It is not cost-cutting and not a missing feature; it is a deliberate boundary.

There is no server-side OpenAI key, no shared pool of credits, and no app-side metering of model usage. The Electron shell bundles the Codex CLI binary per platform/arch. The first-launch wizard spawns `codex login` so the user authenticates against OpenAI directly. Every subsequent `codex exec` call runs against that account at whatever tier the user pays for. The app sees the same auth state Codex sees: a successful login, a rate-limit window, an expired token. Nothing more.

Three properties follow.

1. **No second subscription, ever.** Other AI-study tools layer a marked-up fee on top of an API key the vendor holds. Get It. cannot do that, because it never holds the key in the first place. ChatGPT Plus is the practical floor for sustained study sessions; the free tier signs in but its Codex allowance is intentionally small. Higher tiers give more headroom in the exact same flow.
2. **No data resale and no transit-stage intermediary.** Because we never proxy the model traffic through our infrastructure, there is no Get It. infrastructure for that traffic to flow through. Work-context journals, knowledge graphs, and per-doc folders all live under the user-data directory on local disk. There is no upload step, no opt-in cloud sync, no analytics SDK. "Download your data" is a one-click affordance, but the more honest framing is that there is nothing else *to* download.
3. **The transport is replaceable.** Codex CLI is one of several ways the app could speak to a model. We ship it today because it has the best ergonomics around per-tier login, its bundled binary is small, the official SDK gives us schema-typed responses without DIY enforcement, and it is the only path through which a ChatGPT Plus account can drive a developer-facing CLI without an extra API-key purchase. If a comparable bring-your-own-account transport for another provider appears, `runJson` is the single touchpoint that needs to change.

The same property protects the project legally. **Get It. is not affiliated with OpenAI, not endorsed by OpenAI, not sponsored by OpenAI**, and not a derivative work of any closed-source OpenAI software; it is an independent application that interoperates with the publicly released Codex CLI and uses the end user's own credentials. The student's use of OpenAI's models through Get It. is governed by OpenAI's own Terms of Use, Usage Policies, and Privacy Policy. Those documents are authoritative.

Desktop packaging

The Electron shell is the boring kind of shell: it does as little as possible.

`electron/main.js` acquires a single-instance lock, normalises the user-data directory to `get-it` (overriding Electron's default `Application Support/Get It` so the path matches the pure-Next dev default), runs the setup wizard, spawns the Next.js standalone server as a child Node process on a free localhost port, and points one Chromium `BrowserWindow` at `http://127.0.0.1:<port>`. There is no native menu reinvention, no custom IPC for application data, and no second renderer. The UI is the unchanged Next.js app.

We chose Electron over Tauri because we wanted a guaranteed Chromium runtime on every supported OS: Three.js, KaTeX, the `new Function(...)` LLM sandbox, and pdf.js fonts all behave identically on every machine the user can install on.

`electron/setup.js` owns the Codex life-cycle. The Codex CLI binary ships *inside* the app: it is a Rust binary packaged as an npm optional dependency (`@openai/codex-<platform>-<arch>`) that the SDK locates via `createRequire`. At build time `scripts/electron-prepare.mjs` fetches the correct platform tarball from the npm registry (so a cross-arch build from an Apple Silicon Mac can still produce a usable Windows installer) and stages it under `electron/codex-bin/<triple>/codex/codex(.exe)`. At runtime the setup module resolves that path first; if missing or out of date, an "Install Codex CLI" button downloads it on demand into the user-data dir. The OAuth sign-in is run by spawning `codex login` and capturing the success line from stdout. The wizard is a stand-alone `BrowserWindow` loaded from a plain `file:///` page with a minimal context-isolated preload bridge.

Two boot guards worth knowing about.

- The `window-all-closed` handler **does not auto-quit** while a `bootstrapping` flag is true. Without that flag, dismissing the update modal or the wizard (which are both their own `BrowserWindow`) becomes the *last* open window and the implicit auto-quit fires before `whenReady` can reach `createMainWindow()`. The flag flips to false the instant the main window opens.
- `ELECTRON_RUN_AS_NODE=1` is unset at boot. If that env var leaks in, Electron loads as plain Node and `app` is undefined, which manifests as the cryptic `Cannot read properties of undefined (reading 'requestSingleInstanceLock')`. We catch that case and unset before any API touches `app`.

Build and release pipeline

Multi-target builds run from `scripts/build-electron.mjs`. Locally:

```
node scripts/build-electron.mjs --target=mac-arm64   # or mac-x64 / win-x64 / --all
```

CI: pushing a `v*.*.*` tag to `main` triggers `.github/workflows/release.yml`. The workflow:

1. Rewrites `package.json#version` from the pushed tag so the same number flows into Info.plist / NSIS metadata, into `NEXT_PUBLIC_APP_VERSION` for the in-app version chip, and into the asset filenames.
2. Builds each target on a native runner: macOS Apple Silicon and macOS Intel both run on `macos-latest`, the latter cross-building via `electron-builder --mac --x64` because GitHub's Intel runners (`macos-13`) are being deprecated and queue times are unreliable. Windows builds on `windows-latest`. There are no native modules in the bundle (the standalone server is pure JS, the Codex binary is fetched per target by `electron-prepare.mjs`) so cross-arch is clean.
3. Uploads each artefact to a workflow artifact.
4. A final `publish` job collects them and creates the GitHub Release tied to the tag.

Builds are unsigned. Real signing certificates are a deliberate choice deferred to a sustainable funding moment, not a missing piece of the architecture. The first-launch Gatekeeper / SmartScreen warning is dismissed once and never seen again.

Auto-update

On boot, before the wizard, `electron/updater.js` calls the GitHub Releases API for `beltromatti/get-it`, semver-compares its tag against `app.getVersion()` (the value the CI step pinned), and picks the asset whose filename matches the running platform and arch. When a newer version exists, a polished `BrowserWindow` shows the release notes and an "Update now" button. Clicking it downloads the asset with a live progress bar, hands the file

to `shell.openPath` (Finder mounts the `.dmg`, Windows runs the NSIS installer, Linux surfaces the `.AppImage` to the file manager), and quits so the installer can replace the app on disk.

The user's library, work-context journals, knowledge graphs and settings all live outside the app bundle, so an in-place install never touches them. Network failures, 404s when no release is published yet, and assets missing for the running platform all silently bypass; the rest of startup proceeds unaffected.

Origin and trajectory

Get It. was built in 24 hours at **GDG AI Hack 2026, Milan**, for the **Braynr** challenge. Hackathon team:

- Mattia Beltrami (Politecnico di Milano)
- Matteo Impieri (Politecnico di Milano)
- Filippo Difronzo (Politecnico di Milano)
- Luca Feggi (Università di Padova)

The hackathon submission lived at commit `277ec43` and contained the core architecture this writeup describes: the visualizer pipeline with all five renderer types, the knowledge-graph build agent, the four-axis evaluator, the chat / flashcards / Feynman tools, and the work-context journal. Two design decisions that look obvious in hindsight come straight from the time constraint:

- **`new Function` for the LLM-emitted JS** was the only sandbox we could plausibly ship in 24 hours. We documented it as a defense against LLM mistakes rather than adversarial input; the boundary has held up because the bring-your-own-account model means the user is running their own Codex calls against their own PDFs.
- **Filesystem-only persistence**. Spinning up a database under a hackathon clock would have eaten the time we needed for the evaluator. The JSON-files-under-a-data-dir layout was a deadline call. It then turned out to be the right call once we added the desktop shell: the same files are now what the auto-update flow preserves across version bumps, and the same files are what the user downloads in a click.

Everything beyond `277ec43` is post-hackathon polish that turned the demo into a shipping product. Roughly chronological:

- **Server-side jobs runner.** Detection and per-tag viz generation moved from renderer loops into singleton-per-doc jobs inside the Next process. The viewer became a poll-and-display consumer. Multi-doc parallel progress and reopen-where-you-left-off both fell out for free.
- **Persistent Library** with `lastOpenedAt`, tag-progress and KG-status badges that poll the same job source as the viewer.
- **Desktop shell**. Electron main, embedded server, free-port spawn, single-instance lock. The renderer is byte-identical to the hackathon Next app.
- **First-launch setup wizard.** Bundled Codex binary, OAuth sign-in capture, re-entry on `auth_lost`.
- **Auto-update.** GitHub Releases poll on boot, in-app installer flow, no data loss across version bumps.
- **Codex error classifier + health mailbox**. The four-category banner with retry-deadline countdown, plus the evaluator queue's automatic resume.
- **Quizzes tool** (v1.1.0). The fourth study surface, with `crypto.randomInt`-driven option shuffle so the agent's positional bias does not leak.
- **Cross-arch CI**. Both macOS targets now build on `macos-latest`; the Intel slice cross-compiles.
- **Bring-your-own-account messaging**. The Notice, the writeup section above, the in-app wizard copy: all aligned so the legal posture and the product positioning are the same sentence.

The hackathon clock is no longer a load-bearing constraint, but the product it forced us into has not moved.

What's not here yet

A few choices are deliberately deferred.

Vocal Feynman. The same agent loop and the same end-of-session summary, with a streaming TTS layer over the child voice. The text variant ships today because typed transcripts are strictly better evaluator inputs (no transcription error, no per-token cost, full searchability). The data shape does not change.

A hosted multi-user backend. Out of scope by design. Get It. runs locally against the user's own Codex login, against their own PDFs, on their own machine. The Braynr policy band (source-grounded only, local-first, tiered

access) we get for free at this scale.

Code signing. Unsigned builds today; the first-launch Gatekeeper / SmartScreen warning is dismissed once and not seen again. Real certificates are a funding decision.

Notice and license

Get It. is an independent project. It is not affiliated with, endorsed by, sponsored by, or otherwise associated with OpenAI. The app uses the official open-source Codex CLI as the transport between the local app and OpenAI's models, signed in with the end user's own ChatGPT or OpenAI API account. "OpenAI", "ChatGPT", and "Codex" are trademarks of their respective owner; we use the names only to describe what Get It. interoperates with.

Your use of OpenAI's models through Get It. is subject to OpenAI's own Terms of Use, Usage Policies, and Privacy Policy, and to the Codex CLI's own license and release notes. Those documents are authoritative for what the model service permits and how data is handled on OpenAI's side.